

##Assignment 6: Model a multi-layer neural network model to demonstrate any deep learning application.

Assignment 6: Model a multi-layer neural network model to demonstrate any deep learning application.

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
```

```
# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-11490434/11490434 ————— 0s 0us/step
Training data shape: (60000, 28, 28)
Testing data shape: (10000, 28, 28)
```

```
# Normalize (0-255 → 0-1)
x_train = x_train / 255.0
x_test = x_test / 255.0

# Flatten images (28x28 → 784)
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

# One-hot encoding
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
```

```
plt.imshow(x_train[0].reshape(28,28), cmap='gray')
plt.title("Sample Image")
plt.show()
```

Sample Image



```
model = models.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1
)
```

Epoch 1/10	1688/1688	10s 5ms/step	- accuracy: 0.92
Epoch 2/10	1688/1688	8s 5ms/step	- accuracy: 0.967
Epoch 3/10	1688/1688	8s 4ms/step	- accuracy: 0.977
Epoch 4/10	1688/1688	8s 5ms/step	- accuracy: 0.982
Epoch 5/10	1688/1688	7s 4ms/step	- accuracy: 0.986
Epoch 6/10	1688/1688	8s 5ms/step	- accuracy: 0.987
Epoch 7/10	1688/1688	8s 5ms/step	- accuracy: 0.990
Epoch 8/10	1688/1688	7s 4ms/step	- accuracy: 0.991
Epoch 9/10	1688/1688	8s 5ms/step	- accuracy: 0.992
Epoch 10/10			

1688/1688 ————— 10s 5ms/step - accuracy: 0.99

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
```

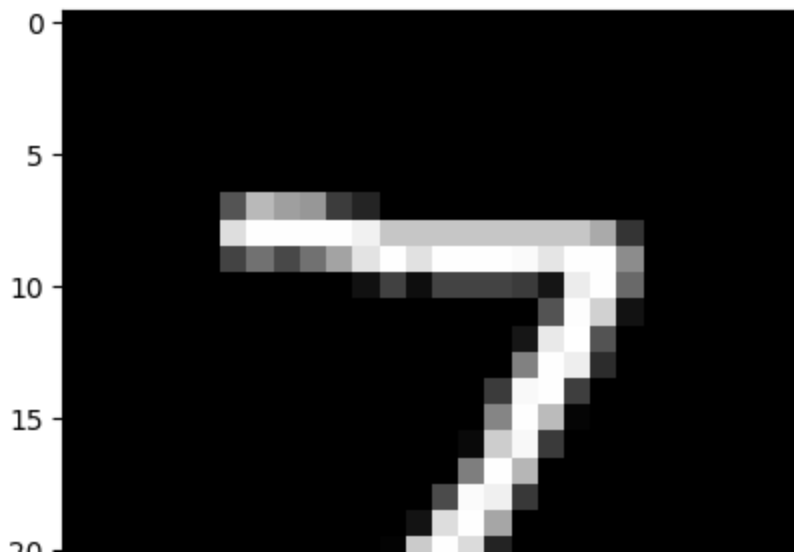
313/313 ————— 1s 3ms/step - accuracy: 0.9775
Test Accuracy: 0.9775000214576721

```
predictions = model.predict(x_test)

# Show prediction
index = 0
print("Predicted:", np.argmax(predictions[index]))
print("Actual:", np.argmax(y_test[index]))

plt.imshow(x_test[index].reshape(28,28), cmap='gray')
plt.show()
```

313/313 ————— 1s 2ms/step
Predicted: 7
Actual: 7



```
# Accuracy
plt.plot(history.history['accuracy'], label='train')
plt.plot(history.history['val_accuracy'], label='validation')
plt.title('Model Accuracy')
plt.legend()
plt.show()

# Loss
```

```
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='validation')
plt.title('Model Loss')
plt.legend()
plt.show()
```

